

Content generation — the strategy

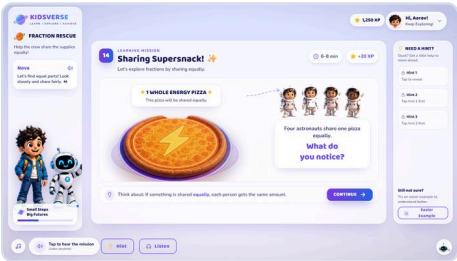
Kidsverse+ — how the three lesson screens take generated content, what they need, and in what order to build it

Answers the content-generation brief Date 8 September 2026 Depth in Content Slots · Content Contract · Content Mapping

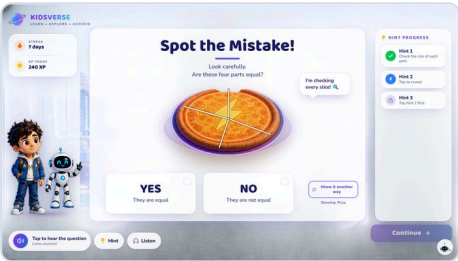
“We need to create a process to generate content for this screen — heading, subheading, explanation, image to explain, hint 1, 2, 3 and script for explanation via Nova — review what all is needed to fulfil this screen via API, so that we can create placeholders on screen to fit any content. First share your strategy.”

Before anything: the screens in the brief are the old design

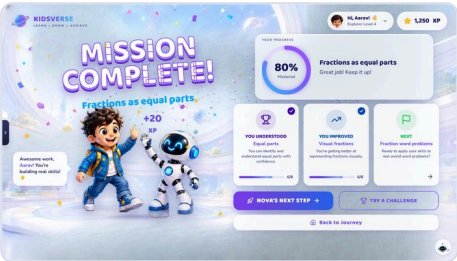
The three screenshots in the brief show a six-stage rail, “Stage 1 of 5”, “Stage 16 of 20”, “Mission Progress 16/20”, “8 of 10 skills mastered” and a bottom dock. **None of these exist any more** — they were fixed numbers that never moved and were removed. Placeholders specified from those screenshots would be holes nothing renders into. The screens as they run today are below; work from these.



14 · Discover — as it runs today



16 · Spot the Mistake — as it runs today



17 · Mission Complete — as it runs today

1. The strategy, in four rules

#	Rule	What it means in practice
1	A screen is a template of named holes.	Every place content appears has a name, a type and a size. The screen never knows what a fraction is; it fills holes. Fractions, Comprehension and Living Things use the same three templates.
2	The holes are a contract, and the contract is one JSON.	The engine produces that JSON; the screen renders it. Not text to be parsed, not fields to be guessed — the exact object in the Content Contract PDF. It is what the Fractions mission runs on today.
3	Do not ask for pictures the app can draw.	For fractions the app draws the pizza, the bar and the number line from a <code>split</code> . The engine sends numbers, not three images, and the picture can never disagree with the answer. Uploads only where there is no renderer.
4	Nothing about the child lives in content.	XP, mastery, progress bars, streaks are the app’s, computed from the child’s record. A content field that describes the learner is a bug — the old “80% mastered” was one, identical for every child.

Status: rules 1–4 are implemented

All three screens read a learning package (`src/content/fractions-equal-parts.json`) through a loader that also accepts a remote endpoint. Rendered output is unchanged pixel for pixel; build passes; zero console errors. The placeholders the brief asks for exist and are filled.

2. The brief lists six things. The screen needs more.

The six named in the brief are the right start. This is the full set, for the Discover screen the brief was written against. Names are the JSON keys.

In the brief	Slot	What the brief did not list, and why it matters
Heading	mission.title	+ mission.code (the “14” badge), mission.emoji
Subheading	mission.subtitle	+ rail.title and rail.blurb — the left column (“Fraction Rescue”)
Explanation	prompt.statement , prompt.question	+ think_about (one emphasised word), model.title , model.caption — four separate texts read as one “explanation”
Image	model.key / split , or an upload	+ crew.count and crew image — the count must equal the number in the sentence
Hint 1, 2, 3	hints[3] , unlock in order	+ easier — the simpler version (two astronauts), on/off with its own sentence
Nova script	nova.voice	nova.speech is a separate field: the bubble text is not the spoken script
—	duration_min , xp	Both are shown as chips; both are authored

Not content: the child’s name, XP total, the “LEARNING MISSION” eyebrow, button labels. The other two screens are itemised the same way in the Content Slots PDF — 14 slots on Spot the Mistake, 9 on Mission Complete, of which 3 are computed.

3. “Via API” — already wired

```
VITE_CONTENT_API=https://api.example.com
GET {VITE_CONTENT_API}/learning-packages/{content_id} → the package, content_id echoed
```

- the bundled package renders on the first frame; the remote one replaces it when it arrives
- any failure keeps the bundled copy — a child never sees an empty screen
- no frontend change to switch sources: set the variable, serve the shape

The engine can be developed against the live app from day one. That is the “why” behind doing the placeholders first: a generator with nothing to render into cannot be checked.

4. “To fit any content” — one condition

Any content, within limits that live in the form

The screens are fixed-layout for a tablet. A title of 32 characters fits; one of 40 wraps onto the picture. Every text slot has a hard character limit and every image a size (Content Slots PDF). Enforce them where the content is typed or generated — a review step catches some of it, a form limit catches all of it. “Any content” means any content of the right size, and the size is part of the contract.

5. Mapping 7 contents and 8 questions onto three screens

Short version; the full tables are in the Content Mapping PDF.

- **Contents** → **Discover**. All seven fill the same slots. What differs is words, picture and default representation. Order = `content_no`.
- **Questions** → **Spot the Mistake**. Seven of eight are “picture, 2–4 options, one right”. One yes/no runs today; six run once the screen takes 2–4 options (one change); the scenario’s free-text “why” needs a widget, or is phrased as pick-one.
- **Sequence = teach** → **check pairs**, joined by `related_content_id`, not seven teaches then eight checks. A miss is caught where it happens; two misses re-open the same content on its alternate picture.
- **Mission Complete is computed**. The engine authors the wording per skill; the app decides which card each skill lands in from the check results.

6. The process, in order

#	Step	State	Why this order
1	Freeze the contract (slots, limits, JSON)	done	Everyone builds to one shape.
2	Screens render from the package	done	The placeholders exist before anything is generated into them.
3	Check screen takes 2–4 options	small	Unlocks 6 of the 8 question types.
4	Engine returns 7 + 8 real items for <i>one</i> topic	engine	Walk it end to end on the real templates before scaling.
5	Pair-walker: play the list as teach→check pairs	small	Needs step 4’s data to be meaningful.
6	Event log + mastery computation	medium	Turns Mission Complete’s numbers real (Data Model PDF).

The reverse order — generation first — produces content no screen can show and no number can check. That is the whole argument for placeholders first, and it is the part that is already finished.

7. Two decisions the engine team needs from product

1. **All seven pairs in one sitting, or a few per day with resume?** Fifteen minutes is long for a young child. The shape supports either; resuming needs the run to be saved.
2. **Scenario questions: free text, or pick-one?** Pick-one is zero frontend work and loses little.